DATA 607
PROJECT REPORT

Alberta Domestic Well Water Quality: Lead Contamination Prediction

Ali Al-Rikabi 10166070
Namya Dimri 30142290
Sushant Kapoor 30296160

Problem Context and Project Objectives

The primary objective of this study is to assess and predict groundwater quality using historical environmental data, with a specific focus on trace heavy metal contamination. Access to safe, uncontaminated drinking water is a critical public health priority, as heavy metals such as lead pose severe health risks even at very low concentrations. The real-world consequences of such contamination have been widely highlighted in public discourse, including in media portrayals such as the Netflix series Lead Children, which depicts the long-term health and societal impacts of lead exposure resulting from industrial activity. This underscores the importance of proactive monitoring and early detection of contamination risks.



Fig. 1: Illustration of industrial pollution and its potential impact on environmental health, inspired by media portrayals such as Lead Children. This highlights the real-world consequences of heavy metal contamination in water systems. [1]

Lead contamination poses severe health risks even at very low concentrations, with no safe level of exposure identified by global health organizations. Recent reporting indicates that approximately one in three children worldwide (around 800 million) have blood lead levels high enough to cause long-term health damage [2].

Traditional approaches to water quality assessment rely on periodic physical sampling and laboratory testing, which, while accurate, are resource-intensive, time-consuming, and often limited in spatial and temporal coverage. In contrast, this study explores a data-driven approach that leverages historical spatial and temporal patterns to estimate contamination risk. By utilizing machine learning models trained on past observations, it becomes possible to identify high-risk wells and prioritize monitoring efforts more efficiently, potentially reducing the need for exhaustive testing across all locations.

This project analyzes a region-specific dataset containing well water sampling records from 2001 to 2020, providing a comprehensive view of groundwater conditions over time. Rather than relying on standard benchmark datasets, this analysis uses original, localized data, yielding insights that are directly relevant to regional environmental health.

The core analytical task is framed as a supervised machine learning classification problem. Instead of predicting the exact concentration of lead (regression), the study adopts a binary classification approach based on established safety thresholds. Specifically, the target variable is defined using a lead concentration cutoff of 0.005 mg/L established by Health Canada [3]. The objective is to develop a predictive model capable of determining whether a given well sample exceeds this critical limit based on its spatial, temporal, and environmental attributes. This formulation aligns closely with regulatory practices, where exceedance of safety thresholds triggers intervention, making the model outputs directly actionable for environmental monitoring and decision-making.

Dataset Overview

The dataset used in this study was obtained from Open Data Alberta's Domestic Well Water Quality in Alberta – Trace Elements dataset [4]. The data consists of multiple annual Excel files spanning from 2001 to 2020, which were cleaned, standardized, and concatenated to form a single consolidated dataset for analysis.

The final dataset contains 8,292 observations and 29 variables, capturing both chemical measurements and contextual information for each well sample.

The complete set of variables is as follows:

SampleID, Date, Source, WellDepth, Aluminum, Antimony, Arsenic, Barium, Beryllium, Boron, Cadmium, Chromium, Cobalt, Copper, Lead, Manganese, Mercury, Molybdenum, Nickel, Selenium, Silver, Thallium, Titanium, Uranium, Vanadium, Zinc, Latitude, Longitude, STRM.

Data Methodology and Analytical Framework

The methodological approach involves a comprehensive pipeline of data ingestion, spatial enrichment, and rigorous model evaluation. The dataset incorporates critical geographical coordinates (Longitude and Latitude) and temporal markers (Sample Date) alongside the chemical analysis results.

Spatial Enrichment and Feature Engineering

A key theoretical component of the data processing is the integration of spatial enrichment. The pipeline utilizes a SPATIAL_K = 10 parameter, indicating the application of a K-Nearest Neighbors (KNN) spatial algorithm. In environmental data, geographic proximity is often highly correlated with contamination levels due to shared aquifers or localized industrial runoff (Tobler's First Law of Geography). By engineering features based on the 10 nearest spatial neighbors, the model can capture localized geospatial patterns and cluster effects that independent coordinate points alone cannot provide.

Model Selection and Theoretical Evaluation

To classify the lead contamination risk, the analysis tests three distinct algorithmic approaches: Logistic Regression, LightGBM (a gradient-boosting framework), and a Multi-Layer Perceptron (MLP) Neural Network. Given the inherent nature of environmental hazard data, the dataset exhibits a severe class imbalance. As shown in figure 2, the negative class (safe levels, n=7687) vastly outnumbers the positive class (hazardous levels, n=605).

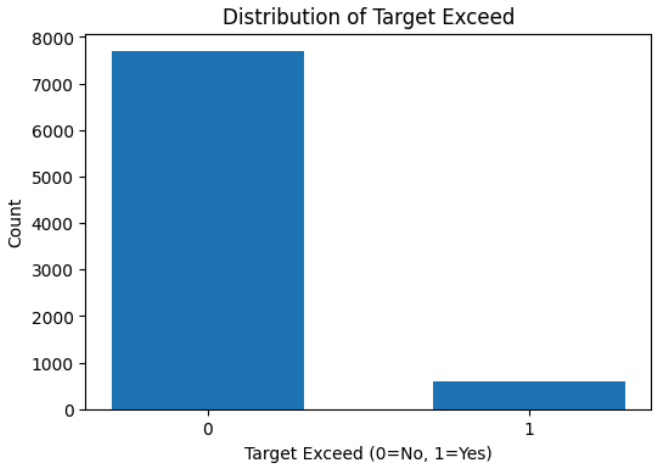


Fig. 2: Distribution of target exceeded showcasing class imbalance

To account for this imbalance, the analysis thoughtfully moves beyond simple accuracy metrics, which can be misleadingly high when a model simply defaults to the majority class, and evaluates the models using a robust suite of theoretical performance metrics:

- **Precision and Recall (F1-Score):** The analysis calculates these metrics to evaluate the trade-off between false positives and false negatives. In this public health context, *recall* (the ability to correctly identify all actual hazardous wells) is arguably the most critical metric, as missing a contaminated well poses a severe risk. The current Neural Network results indicate a heavy bias toward the majority class (Recall of 0.00 for the positive class), highlighting the challenge of the skewed data.

- **ROC-AUC (Receiver Operating Characteristic - Area Under Curve):** This metric evaluates the model's ability to discriminate between the safe and hazardous classes across all possible classification thresholds. An AUC of 0.5 represents a completely random guess, which aligns closely with the baseline Logistic Regression and Neural Network results currently observed in the analysis.

- **PR-AUC (Precision-Recall Area Under Curve):** Because ROC-AUC can sometimes present an overly optimistic view of highly imbalanced data, the analysis rightly calculates the PR-AUC. This metric focuses strictly on the minority (hazardous) class, providing a much more accurate theoretical reflection of how well the model predicts rare contamination events.

- **Brier Score Loss:** Finally, the analysis incorporates the Brier Score to measure the calibration of the predicted probabilities. Theoretically, the Brier Score calculates the mean squared difference between the predicted probability of contamination and the actual binary outcome. A lower score (closer to 0) indicates that when the model predicts a 10% chance of contamination, the outcome is statistically accurate. This ensures that the model's risk assessments are reliable for actual environmental decision-making.

Given the temporal nature of the dataset, a time-based train-test split was used instead of a random split. Data from 2001 to 2019 was used for training, while samples from 2020 were reserved for testing [see Appendix figure a and figure b]. This approach ensures that the model is evaluated on future, unseen data, better reflecting real-world deployment where predictions are made on new observations. It also prevents temporal data leakage that could occur if information from future samples were inadvertently included during training.

Baseline Model: Logistic Regression

Logistic Regression was used as an initial baseline model due to its simplicity, interpretability, and strong performance on structured tabular data. Two versions of the model were evaluated to highlight the impact of feature selection and potential data leakage.

Initial Model with Full Feature Set

The first implementation of Logistic Regression utilized the full set of available features, including concentrations of other heavy metals such as aluminum, arsenic, cadmium, and zinc, alongside spatial and temporal variables. Numerical features were standardized, categorical features were one-hot encoded, and missing values were imputed using median (numerical) and most frequent (categorical) strategies. Class imbalance was addressed using balanced class weights.

```
=== Logistic Regression Results ===
ROC-AUC: 0.9999763565433266
PR-AUC: 0.9996753246753247
Brier Score: 0.007493079217129693

Confusion Matrix:
[[764   5]
 [  0  55]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      0.99      1.00       769
           1       0.92      1.00      0.96        55

    accuracy                           0.99       824
   macro avg       0.96      1.00      0.98       824
weighted avg       0.99      0.99      0.99       824
```

Fig. 3: Logistic Regression results showcasing perfect performance due to data leakage

This model achieved near-perfect performance as shown in figure 3, with a ROC-AUC of approximately 1.00 and a PR-AUC close to 1.00, alongside a very low Brier score. At first glance, these results suggest an accurate classifier capable of perfectly distinguishing between safe and unsafe water samples.

However, this performance is misleading. The inclusion of other metal concentrations introduces data leakage particularly those that are chemically or environmentally correlated with lead. In practice, these variables may act as proxies for the target itself, allowing the model to infer lead exceedance indirectly rather than learning meaningful predictive patterns. As a result, the model's performance is unrealistically optimistic and would not generalize to real-world scenarios where such information may not be available at prediction time.

Refined Model with Restricted Feature Set

To address this issue, a second Logistic Regression model was trained using a restricted set of "safe" features: geographic location (latitude and longitude), well depth, source type, and temporal variables (year and month). Additionally, a spatial feature (spatial_k) was introduced to capture local contamination patterns based on nearby wells as visualized in figure 4.

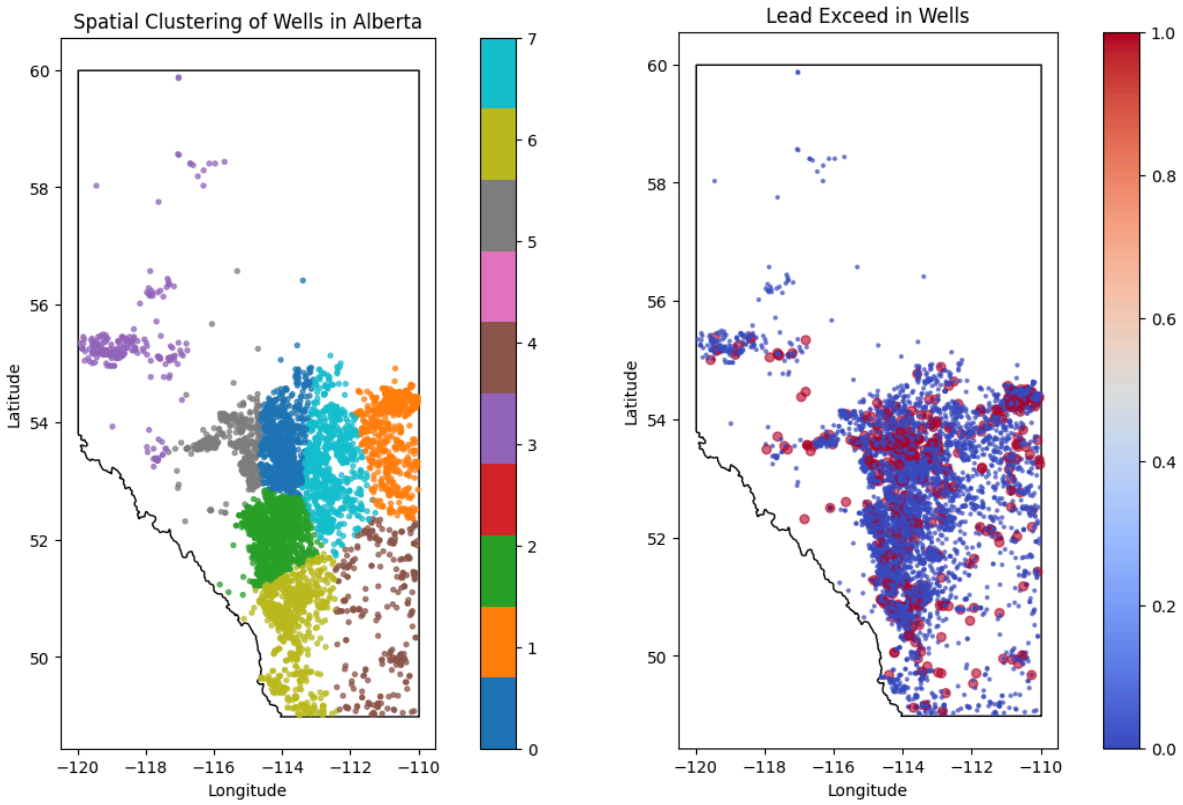


Fig. 4: Spatial Clustering of Wells using spatial clustering (left), Lead concentration exceeded spatial visualization (right)

This refined model represents a more realistic deployment scenario, where predictions must be made using readily available environmental and contextual information rather than direct chemical measurements.

```
=== Logistic Regression Results ===
ROC-AUC: 0.5518619222130275
PR-AUC: 0.10135011087970783
Brier Score: 0.20758710857175505

Confusion Matrix:
[[628 141]
 [ 40  15]]

Classification Report:
              precision    recall  f1-score   support

           0       0.94      0.82      0.87       769
           1       0.10      0.27      0.14        55

    accuracy                           0.78       824
   macro avg       0.52      0.54      0.51       824
weighted avg       0.88      0.78      0.83       824
```

Fig. 5: Logistic Regression results after reducing features and introducing spatial features

Under this setting, performance dropped significantly (see figure 5), with ROC-AUC around 0.55 and PR-AUC around 0.1, indicating only marginal improvement over random guessing. The confusion matrix further shows that while the model is able to identify some positive cases, it does so with very low precision, leading to a high number of false positives.

Key Observations

The comparison between the two models highlights a critical insight:

- The initial model's near-perfect performance is driven by leakage through highly correlated chemical features, rather than true predictive capability.
- When restricted to realistic features, Logistic Regression struggles to effectively separate classes, suggesting that the available inputs provide limited predictive signal.
- This reinforces the importance of careful feature selection and validates the decision to adopt a more constrained feature set for subsequent models.

Role as a Baseline

The refined Logistic Regression model is treated as the true baseline for this study. It provides a transparent and interpretable benchmark against which more complex models (e.g., gradient boosting and neural networks) can be evaluated under realistic conditions.

Gradient Boosting Models: LightGBM and XGBoost

To explore whether more advanced, non-linear models could improve performance, two gradient boosting algorithms LightGBM and XGBoost were evaluated using the same feature set as the refined Logistic Regression model. These models are well-suited for tabular data and are often effective at capturing complex interactions between features.

LightGBM

LightGBM was trained using the original feature set with categorical variables handled natively through categorical encoding. The model achieved a ROC-AUC of approximately 0.55 and a PR-AUC of 0.08, representing a slight improvement over Logistic Regression in terms of discrimination (see figure 6). However, performance on the minority class remained weak, with low precision (~ 0.07) and recall (~ 0.25), indicating difficulty in accurately identifying positive cases.

```
=== LightGBM Results ===
Accuracy: 0.7390776699029126
ROC-AUC: 0.5459274145880127
PR-AUC: 0.07963935124821957
Brier Score: 0.1637715230025795

Confusion Matrix:
[[595 174]
 [ 41  14]]

Classification Report:
              precision    recall  f1-score   support

           0       0.94      0.77      0.85       769
           1       0.07      0.25      0.12        55

    accuracy                           0.74       824
   macro avg       0.51      0.51      0.48       824
weighted avg       0.88      0.74      0.80       824
```

Fig. 6: LightGBM results

XGBoost

XGBoost was implemented as a comparative boosting model, using one-hot encoding for categorical variables and a class imbalance adjustment via scale_pos_weight. The model achieved a ROC-AUC of approximately 0.54 and a PR-AUC of 0.078, performing similarly to LightGBM. While XGBoost produced a lower Brier score (~ 0.13), indicating better probability calibration, it showed lower recall (~ 0.16), suggesting a more conservative prediction behavior with fewer positive predictions.

```
=== XGBoost Results ===
ROC-AUC: 0.535855302045159
PR-AUC: 0.07771053994540761
Brier Score: 0.12736881479870046

Confusion Matrix:
[[675  94]
 [ 46   9]]

Classification Report:
              precision    recall  f1-score   support

           0       0.94      0.88      0.91       769
           1       0.09      0.16      0.11        55

    accuracy                           0.83       824
   macro avg       0.51      0.52      0.51       824
weighted avg       0.88      0.83      0.85       824
```

Fig. 7: XGBoost Results

Both LightGBM and XGBoost produced comparable results (see figure 6 and figure 7), with neither model significantly outperforming the other. While LightGBM achieved slightly higher recall, XGBoost provided better-calibrated probability estimates. Overall, however, both models only marginally improved upon Logistic Regression.

This suggests that, despite their ability to model non-linear relationships, the primary limitation lies in the feature set rather than model complexity. The weak performance across all models indicates limited separability between classes under the available spatial and temporal features.

Neural Network (MLP)

A Multi-Layer Perceptron (MLP) was explored to evaluate whether a neural network could capture non-linear relationships in the data that simpler models might miss. Two versions of the model were implemented: a baseline network and an enhanced version with additional tuning and imbalance handling.

Baseline MLP

The initial MLP consisted of two hidden layers (64 and 32 neurons) and was trained using only numerical features. Despite its ability to model complex patterns, the model performed poorly, with a ROC-AUC of approximately 0.50 and PR-AUC of 0.076, indicating no meaningful discrimination ability.

The confusion matrix, shown in figure 8, revealed a critical issue: the model predicted only the majority class, failing to identify any positive cases. This resulted in zero recall for the minority class, despite high overall accuracy (93%), which is misleading in the presence of class imbalance.

```
=== Neural Network Results ===
ROC-AUC: 0.49667809433739213
PR-AUC: 0.07570000325936732
Brier Score: 0.11078005821168595

Confusion Matrix:
[[766   3]
 [ 55   0]]

Classification Report:
              precision    recall  f1-score   support

           0       0.93      1.00      0.96       769
           1       0.00      0.00      0.00        55

    accuracy                           0.93       824
   macro avg       0.47      0.50      0.48       824
weighted avg       0.87      0.93      0.90       824
```

Fig. 8: MLP results showcasing weak discrimination ability of model

This behavior highlights a key limitation of neural networks in this context, without appropriate handling of imbalance and strong feature signals, the model defaults to predicting the dominant class.

Improved MLP

To address these limitations, a deeper network architecture was implemented (see Figure 9), consisting of multiple fully connected layers (128, 64, 32, and 16 neurons) that progressively transform the input features through non-linear activations (see Appendix figure h). This layered structure allows the model to learn increasingly abstract representations of the data. Along with this, feature scaling, SMOTE-based oversampling, and threshold adjustment were applied. This improved model achieved a ROC-AUC of approximately 0.55, showing modest improvement over the baseline.

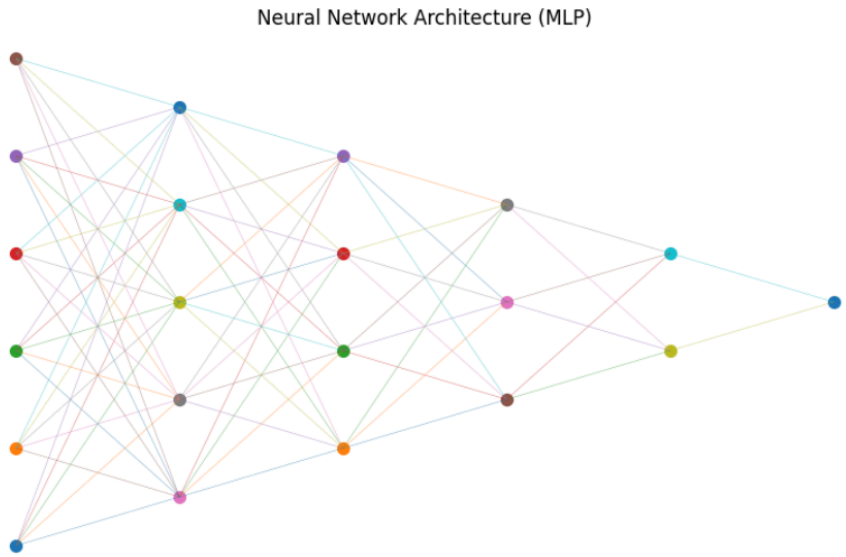


Fig. 9: Improved MLP Network Architecture (simplified version)

Additionally, key hyperparameters such as the learning rate, regularization strength (alpha), and batch size were manually tuned to improve training stability and generalization. The classification threshold was also adjusted to better capture minority class instances. While these adjustments led to slight improvements, their overall impact remained limited.

```
=== Neural Network Results ===
ROC-AUC: 0.5506324624660125
PR-AUC: 0.07932126037871899
Brier Score: 0.09224059538741718

Confusion Matrix:
[[727  42]
 [ 52   3]]

Classification Report:
              precision    recall  f1-score   support

           0       0.93      0.95      0.94       769
           1       0.07      0.05      0.06        55

    accuracy                           0.89       824
   macro avg       0.50      0.50      0.50       824
weighted avg       0.88      0.89      0.88       824
```

Fig. 10: Improved MLP with enhanced discrimination ability compared to baseline MLP

However, performance on the minority class remained weak as shown in figure 10. While the model was able to identify a small number of positive cases, recall (~ 0.05) and precision (~ 0.07) remained low. This indicates that, despite additional complexity, resampling, and tuning efforts, the model still struggled to generalize meaningful patterns for contamination prediction.

Results and Discussion

As shown in table 1, model performance remained modest across all approaches, with ROC-AUC values between 0.49 and 0.55 and PR-AUC values around 0.075-0.10. Logistic Regression and the multi-layer MLP achieved the highest ROC-AUC (0.55), but differences between models were minimal.

Model	ROC-AUC	PR-AUC	Brier Score
Logistic Regression	0.551862	0.101350	0.207587
LightGBM	0.545927	0.079639	0.163772
XGBoost	0.535855	0.077711	0.127369
MLP 1 Layer	0.496678	0.075700	0.110780
MLP Multi-Layer	0.550632	0.079321	0.092241

Table 1: Model Comparison

PR-AUC scores were consistently low, indicating poor detection of the minority class (lead exceedances). Although the multi-layer MLP achieved the best Brier score (0.09), suggesting better probability calibration, this did not translate into improved classification performance.

Overall, increasing model complexity did not yield meaningful improvements. Logistic Regression performed similarly to LightGBM, XGBoost, and MLP models, suggesting that the available features limit predictive performance more than model choice.

Limitations

Several key limitations likely contributed to the observed results:

- Class Imbalance: The dataset contains a small proportion of positive cases (7%), making it difficult for models to learn meaningful patterns for the minority class.
- Feature Constraints: To avoid data leakage, only spatial, temporal, and basic well characteristics were used. While necessary, this restriction likely removed highly predictive chemical features.
- Weak Predictive Signal: The consistently low PR-AUC across all models suggests limited separability between contaminated and non-contaminated wells using the available features.
- Temporal Distribution Shift: A large proportion of samples originate from a single year (2019), which may introduce differences between training and testing distributions.

Conclusion

This study evaluated multiple machine learning models for predicting groundwater contamination using spatial and temporal features. All models showed similar, limited performance, indicating that data and feature quality, and not model complexity, are the primary constraints.

While spatial feature engineering provided useful context, it was insufficient to fully capture contamination patterns. Future work should focus on incorporating richer environmental features and improving data balance to enhance predictive performance.

REFERENCES

[1] *Lead Children*. 2026 Netflix
[2] *The Guardian*, “One in three children have dangerous levels of lead in their blood,” Jul. 30, 2020. [Online]. Available: https://www.theguardian.com/environment/2020/jul/30/one-in-three-children-dangerous-levels-lead-in-blood
[3] Health Canada. (2019). *Water Talk - Lead in drinking water*. [Online]. Available: https://www.canada.ca/en/health-canada/services/environmental-workplace-health/reports-publications/water-quality/water-talk-minimizing-exposure-lead-drinking-water-distribution-systems.html#s3
[4] Domestic Well Water Quality in Alberta - Trace Elements. Open Data. Available: https://open.alberta.ca/opendata/domestic-well-water-quality-in-alberta

APPENDIX

Year	count	fraction	cumulative_fraction
2001	2	0.000241	0.000241
2002	25	0.003015	0.003256
2003	31	0.003739	0.006995
2004	16	0.001930	0.008924
2005	23	0.002774	0.011698
2006	22	0.002653	0.014351
2007	82	0.009889	0.024240
2008	33	0.003980	0.028220
2009	35	0.004221	0.032441
2010	37	0.004462	0.036903
2011	58	0.006995	0.043898
2012	44	0.005306	0.049204
2013	104	0.012542	0.061746
2014	406	0.048963	0.110709
2015	860	0.103714	0.214424
2016	177	0.021346	0.235769
2017	110	0.013266	0.249035
2018	1008	0.121563	0.370598
2019	4395	0.530029	0.900627
2020	824	0.099373	1.000000

Fig. a: Row Count per year

```
cutoff_year = 2019

# Boolean masks
train_idx = df["Year"] <= cutoff_year
test_idx  = df["Year"] > cutoff_year

X_train = X.loc[train_idx].copy()
X_test  = X.loc[test_idx].copy()
y_train = y.loc[train_idx].copy()
y_test  = y.loc[test_idx].copy()

# Recompute column types based on training set
numeric_cols = X_train.select_dtypes(include=[np.number]).columns.tolist()
categorical_cols = X_train.select_dtypes(include=["object"]).columns.tolist()
```

Fig. b: Train Test split based on year

```
# Preprocessing
numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_cols),
        ("cat", categorical_transformer, categorical_cols)
    ]
)

# Baseline model
log_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(
        max_iter=2000,
        class_weight="balanced",
        random_state=RANDOM_SEED
    ))
])
```

Fig. c: Preprocessing Pipeline for Logistic Regression

```
train_coords = X_train[["Latitude", "Longitude"]].values
test_coords  = X_test[["Latitude", "Longitude"]].values

knn = NearestNeighbors(n_neighbors=SPATIAL_K)
knn.fit(train_coords)

# TRAIN spatial feature
_, train_idx = knn.kneighbors(train_coords)
train_spatial = y_train.values[train_idx].mean(axis=1)

# TEST spatial feature (neighbors from TRAIN ONLY)
_, test_idx = knn.kneighbors(test_coords)
test_spatial = y_train.values[test_idx].mean(axis=1)

# Add to datasets
X_train["spatial_k20"] = train_spatial
X_test["spatial_k20"]  = test_spatial
```

Fig. d: Spatial Feature Extraction

```python
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(10, 8))

# Alberta boundary
alberta.plot(ax=ax, color="white", edgecolor="black")

# Clustered wells
geo_df.plot(
    ax=ax,
    column="cluster",
    cmap="tab10",
    markersize=8,
    alpha=0.7,
    legend=True
)

plt.title("Spatial Clustering of Wells in Alberta")
plt.xlabel("Longitude")
plt.ylabel("Latitude")

plt.show()
```

```python
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(10, 8))

# Alberta boundary
alberta.plot(ax=ax, color="white", edgecolor="black")

sizes = np.where(geo_df["target_exceed"] > threshold, 30, 4)

geo_df.plot(
    ax=ax,
    column="target_exceed",
    cmap="coolwarm",
    markersize=sizes,  # Pass the array of sizes here
    alpha=0.6,
    legend=True
)

plt.title("Lead Exceed in Wells")
plt.xlabel("Longitude")
plt.ylabel("Latitude")

plt.show()
```

Fig. e: Spatial Clusters Visualization (left), Target exceed visualization (right)

```python
# Convert categorical columns to 'category'
for col in categorical_cols:
    X_train[col] = X_train[col].astype("category")
    X_test[col] = X_test[col].astype("category")

lgb_model = LGBMClassifier(
    n_estimators=300,
    learning_rate=0.05,
    max_depth=6,
    random_state=RANDOM_SEED,
    class_weight="balanced"
)

# Fit
lgb_model.fit(X_train, y_train)

# Predict
y_pred_lgb = lgb_model.predict(X_test)
y_prob_lgb = lgb_model.predict_proba(X_test)[:, 1]

# Metrics
print("=== LightGBM Results ===")
print("Accuracy:", accuracy_score(y_test, y_pred_lgb))
print("ROC-AUC:", roc_auc_score(y_test, y_prob_lgb))
print("PR-AUC:", average_precision_score(y_test, y_prob_lgb))
print("Brier Score:", brier_score_loss(y_test, y_prob_lgb))
```

```python
# One-hot encode categorical variables
X_train_enc = pd.get_dummies(X_train, columns=categorical_cols, drop_first=True)
X_test_enc = pd.get_dummies(X_test, columns=categorical_cols, drop_first=True)

# Align columns (important!)
X_train_enc, X_test_enc = X_train_enc.align(X_test_enc, join='left', axis=1, fill_value=0)

# Handle class imbalance
scale_pos_weight = (len(y_train) - sum(y_train)) / sum(y_train)

xgb_model = XGBClassifier(
    n_estimators=300,
    learning_rate=0.1,
    max_depth=6,
    random_state=RANDOM_SEED,
    scale_pos_weight=scale_pos_weight,
    use_label_encoder=False,
    eval_metric="logloss"
)

# Fit
xgb_model.fit(X_train_enc, y_train)

# Predict
y_pred_xgb = xgb_model.predict(X_test_enc)
y_prob_xgb = xgb_model.predict_proba(X_test_enc)[:, 1]
```

Fig. f: LightGBM (left), XGBoost (right)

```python
mlp_model = MLPClassifier(
    hidden_layer_sizes=(64, 32),
    activation="relu",
    solver="adam",
    max_iter=300,
    random_state=RANDOM_SEED
)

# Fit
mlp_model.fit(X_train.select_dtypes(include=[np.number]), y_train)

# Predict
y_pred_mlp = mlp_model.predict(X_test.select_dtypes(include=[np.number]))
y_prob_mlp = mlp_model.predict_proba(X_test.select_dtypes(include=[np.number]))[:, 1]

# Metrics
print("=== Neural Network Results ===")
print("ROC-AUC:", roc_auc_score(y_test, y_prob_mlp))
print("PR-AUC:", average_precision_score(y_test, y_prob_mlp))
print("Brier Score:", brier_score_loss(y_test, y_prob_mlp))
```

Fig. g: Baseline Neural Network

```python
mlp_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("mlp", MLPClassifier(
        hidden_layer_sizes=(128, 64, 32, 16),  # deeper network
        activation="relu",
        solver="adam",
        alpha=1e-5,                # regularization
        batch_size=64,
        learning_rate_init=0.001,
        max_iter=500,
        early_stopping=True,
        validation_fraction=0.1,
        n_iter_no_change=20,
        random_state=RANDOM_SEED
    ))
])

# Fit
from imblearn.over_sampling import SMOTE

X_train_num = X_train.select_dtypes(include=[np.number])
X_test_num = X_test.select_dtypes(include=[np.number])

smote = SMOTE(random_state=RANDOM_SEED)
X_resampled, y_resampled = smote.fit_resample(X_train_num, y_train)
mlp_pipeline.fit(X_resampled, y_resampled)
```

Fig. h: Improved Neural Network